

The hurdy-gurdy Platform

Exploring the Frontier of Reducible Decidability in Practice

Christoph Kirsch

University of Salzburg, Austria

Czech Technical University, Prague, Czechia

ck@cs.uni-salzburg.at

Abstract

We are constructing a platform for exploring the *frontier of reducible decidability in practice* — which questions about programs can be answered, today, by chains of checked reductions to mechanized decision procedures within declared budgets — and, eventually, for pushing it. This paper describes the platform designed fresh, after its first campaign, around one organizing move: *translation and solving are the same kind of edge, and results are the only currency*. A solver is a pair whose target is the language of results, so translating is solving; a result is a replayed witness, a universal claim up to a bound with an optional certificate, or an honest partial description of how far a route got and where it failed; and the frontier of a benchmark is exactly the set of its questions whose best result is not terminal — no separate bookkeeping for failure exists. An LLM plays a pinned benchmark autonomously until a human pulls the plug, conjecturing in a fixed order that is the vision’s core discipline — new decision procedures first, then new translations, then new languages: semantics first, then syntax — and everything it builds passes one gate: measured determinism, square or result validity, and two-sided negative controls. The trust story is one sentence: the LLM never writes a result; only the kernel does, by running checked code. The kernel is small, hand-written, and mechanized in Lean 4 — the result order is strict, best-per-question ratchets under log growth, terminal questions stay terminal, grades only climb — and everything else is generated content, with the previous generation of the platform as its first instance, carried over through the gate it now must pass. We report exactly the experiments the redesign rests on: the campaign whose structural lesson — a loop producing only meaning-preserving artifacts is conservative over truth — forced the redesign, and a first demonstration whose three-question map holds a replayed witness, a bounded claim graded exactly as strong as its evidence, and an unbounded ask honestly on the frontier.

1 Introduction

We are interested in constructing a platform that enables us to explore the frontier of *reducible decidability in practice* — and, eventually, to push it. The phrase is chosen word by word. A question about a program is *reducibly decidable* when some finite chain of checked reductions carries it to a mechanized decision procedure that answers it; the reachable

set is the closure of the questions under the reductions one actually has, not an oracle’s reach. And *in practice* means within declared budgets, measured and never asymptotic: a procedure that exceeds every budget is not a way, and where that boundary lies is an empirical fact that moves with every solver, abstraction, and logic the world supplies. The frontier, so understood, is a real and interesting object: on its near side, everything formal methods can do today, enumerated and priced; on its far side, structured evidence about exactly what is missing.

An earlier design of hurdy-gurdy explored this frontier as a cartography problem, with a five-condition answerability filtration, a typed demand ledger, six facilitation theorems, and a human registration valve; its calculus and measured snapshot are the subject of a companion paper [11], and its first campaign ran to completion (Section 7). The campaign taught two lessons. Structurally: a loop whose every admissible artifact is meaning-preserving is *conservative over truth* — it discovers instruments, never facts — so after seven iterations over a suite native to its best solver, the loop’s entire demand output read *get a better engine*. And architecturally: the machinery that tracked demand — books, boards, briefs, mandates, lanes, the filtration — had each earned its place and jointly grown past its justification, because its consumer changed. Demand used to feed mechanical derivation, which needs typed taxonomies; it now feeds an LLM conjecturing, which wants rich evidence and tolerates prose.

This paper describes the platform designed fresh from those lessons, around one organizing move:

Translation and solving are the same kind of edge, and results are the only currency.

A *solver pair* is a pair whose target is the language of results: translating *is* solving under a declared budget (Section 2). A *result* is a replayed witness, a universal claim up to a bound (or unbounded, with a certificate), or an honest *partial* — a description of how far a route got and where it failed. The *frontier of a benchmark* is exactly the set of its questions whose best result is not terminal (Section 4) — no separate bookkeeping for failure exists. An LLM plays a pinned benchmark autonomously until a human pulls the plug, conjecturing in a fixed order that is the vision’s core discipline — new decision procedures first, then new translations, then new languages: *semantics first, then syntax* (Section 5) — and everything it builds passes one gate. The trust

story is one sentence: **the LLM never writes a result; only the kernel does, by running checked code.**

The kernel — the fixed, hand-written part — is deliberately small enough to read in an afternoon, and its properties are mechanized in Lean 4: the result order is strict, best-per-question is a ratchet under log growth, terminal questions stay terminal, grades only climb (Section 6). Everything else is generated content admitted through the gate, and the previous generation of the platform is the first such content: its languages, pairs, and engines are carried over by wrapping them in kernel manifests and re-certifying them through the gate they must now pass (Section 7).

Contributions.

- One edge kind: solver pairs as translation into a result language, with the commuting square degenerating to result validity, so translations and decision procedures share one admission gate (Section 2).
- Results as the only currency: a fixed result schema whose order (level, bound, grade) makes the frontier a derived object — the non-terminal best results — and monotone exploration a property of the data structure, not a theorem apparatus (Section 4).
- The certification symmetry: both existential and universal answers are carry-back-then-check, witness replay mirrored by certificate re-discharge, with a strict grade ladder *claimed* < *checked* < *certified* whose top rung is reserved for route-independent evidence (Section 3).
- The autonomous loop: no registration valve, one gate with measured determinism and two-sided controls, a fixed semantics-first conjecture order, plug-pull safety at any moment, and bootstrap from an empty instance (Section 5).
- The kernel mechanized, and an honest accounting of the previous design: where each of its six facilitation theorems went — proved smaller, made structural, or retired with the machinery that needed it (Section 6).
- Exactly the experiments the redesign rests on: the campaign that forced it, and the carried-over demonstration whose three-question map reproduces the campaign’s structural finding in miniature (Section 7).

2 Languages, pairs, results

Languages. A *language* is a deterministic syntax plus a deterministic interpreter exposing named *observables*. Two kinds. *Root* languages — the formats benchmarks arrive in — are the trusted base: their interpreters are graded *stipulated* until corroborated by benchmark labels or supplied test vectors. Only roots cost trust. A *derived* language is always registered together with a pair to a parent — an *abstraction* (over-approximating) or a *specialization* (a fragment with an exact embedding) — so its semantics is checked against the parent’s and it adds nothing to the trusted base.

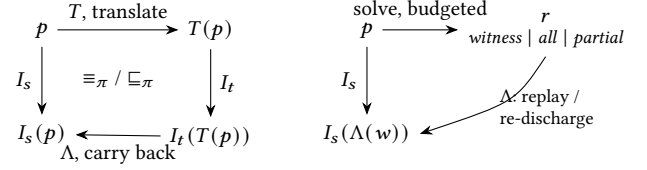


Figure 1. One kind of edge. Left: a translation pair closes a directional commuting square, checked per program by running both sides. Right: a solver pair is the same edge with the result language as target — translating is solving — and the square degenerating to result validity: a witness is carried back and replayed through the source interpreter; a universal certificate is carried back and re-discharged; a partial makes no claim to check. Both kinds pass one gate.

Pairs. The unit of the platform is the *pair*: a directed edge with a pure translator T , a carry-back map Λ , kept observables π , a direction (exact / over / under), measured cost, and a lineage declaration. There are two kinds of pair and one gate (Fig. 1):

- A *translation pair* (language $\rightarrow$ language) is checked per program by the directional commuting square, $I_s(p) \equiv_\pi \Lambda(I_t(T(p)))$: interpreting the source agrees, on π , with translating, interpreting the translation, and carrying the behavior back. An abstraction pair declares $\sqsubseteq_\pi$ instead — every source behavior has a target counterpart, and the target may deliberately have more — universal verdicts transfer back across such a hop, and existential ones only ever return by replay. The square is decidable per program: run both sides, compare under π ; a failure names its step and observable.
- A *solver pair* (language $\rightarrow$ results) has the result language as its target: T is solving under a declared budget. The square degenerates to *result validity* — witnesses must replay at the source, certificates must re-discharge, supplied negative controls must fail, determinism must hold — under the same admission discipline. There is no separate solver gate.

Pairs compose into *routes*, and every playable route ends in a solver pair. A route’s contract is the componentwise meet of its hops’ contracts — the weakest hop on every axis at once — the one composition rule kept from the previous calculus [11]: a route’s label never overstates.

Results. A result is a fixed kernel schema with domain-specific payloads:

$$\begin{aligned} \text{result} &= \{ q, \text{route}, \text{budget}, \text{grade}, \text{value} \} \\ \text{value} &= \text{witness}(w, \text{depth}) \\ &\quad | \text{all}(\text{bound} : k \mid \infty, \text{cert}?) \\ &\quad | \text{partial}(\text{evidence}) \end{aligned}$$

witness says *here is a w* — and a recorded witness is always *replayed* (Section 3); a solver’s unreplayable *sat* is never a witness result, only evidence inside a partial. *all* says *none exists up to the bound* (or unboundedly, certificate permitting). *partial* is deliberately semi-structured: a small typed core the kernel orders on — bound reached, budget spent — plus free-form progress description and solver profiling, because its reader is the conjecturing LLM, not a derivation engine. This is the rule that keeps the redesign’s collapse honest: **structure only what the kernel must compute with; everything else is evidence and stays prose.** Budgets ride in every result’s provenance; capped is labeled capped; cost is recorded and reported, never ranked.

Determinism, measured. Every registered executable — interpreter, translator, solver shim — is a pure deterministic function from bytes to bytes, run sandboxed (own process, empty environment, temporary working directory, wall and memory caps). The kernel does not believe this; it measures it: *every check runs twice and byte-compares the output*. Determinism is the load-bearing wall — it is what makes the square decidable per program, replay evidence rather than anecdote, and every map entry re-derivable.

3 Certification and the grade ladder

Both certifications are carry-back-then-check — Λ , then a run — and they are mirrors (Fig. 2); the difference is what checks.

Existential. A witness is carried back through each hop’s Λ and *replayed* through the source interpreter: one run, always available, route-independent. A wrong or adversarial pair cannot forge it — the interpreter that decides is the trusted base, not anything the route’s author wrote (existential_self_certifying in the companion calculus [11], axiom-free). On an abstraction route a witness that fails replay is a *spurious counterexample*: not a bug but a refinement demand, recorded inside a partial.

Universal. A certificate is interpreted *against the problem* — the checker is a function of the problem and the certificate, never of the solver’s say-so. Checked at the *target*, the claim inherits the route’s trust (the meet). Carried back and *re-discharged at the source* — finitely many decidable obligations; for an inductive invariant: base, step, safe — it becomes route-independent, the exact mirror of witness replay. The fail-safe direction is definitional: a wrong or wrongly-carried certificate can only fail to upgrade a claim, never fake one.

The kernel computes grades from lineage declarations, and grades only improve (Section 6). Nothing is ever worded stronger than what was verified. One intended second-order effect: whatever earns the top grade is what the autonomous loop learns to build, so the frontier evidence itself pushes the LLM toward certificate printers, carry-back maps for certificates, and independent discharge engines.

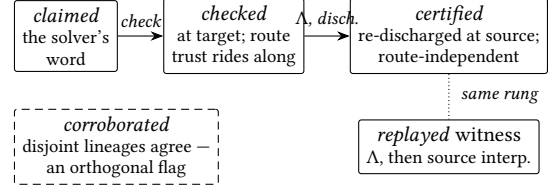


Figure 2. The strict grade ladder. Universal claims climb *claimed* < *checked* < *certified*; *certified* is reserved for the route-independent form, and every result states its rung and what its residual trust rests on — the route meet for *checked*, the discharge lineages for *certified*. Existential answers do not climb: a recorded witness is always *replayed*, which is the same carry-back-then-check evidence as re-discharge and shares its rung. *Corroborated* — agreement across disjoint declared lineages — is orthogonal to the ladder.

Contradictions. A replayed witness is ground truth. If a question ever holds both a replayed witness and a covering universal claim, the kernel records a *contradiction event*: the witness stands, the universal’s certification chain is marked falsified, and nothing is silently resolved. Contradiction events are the platform’s fire alarm — the one record that says a graded chain lied — and their permanence is what makes the grades meaningful.

4 The frontier is the non-terminal results

A *question* is $q = (\text{language}, p, \text{observable}, \text{mode}, \text{bound})$ with mode existential or universal and bound k or ∞ ; a *benchmark* B is a pinned finite set of questions with recorded provenance — sha256 per program, labels where they exist — pinned so that “the benchmark” names the same set at every iteration.

The order. Results for a question are ordered by the key (*level, bound, grade*), lexicographically:

$$\underbrace{\text{partial}}_{\text{level 0}} < \underbrace{\text{all}(k) \text{ below the ask}}_{\text{level 1}} < \underbrace{\text{terminal}}_{\text{level 2}}$$

where *terminal* is a replayed witness or a universal claim covering the asked bound; within a level, higher bound wins, then higher grade. Cost is deliberately not in the key: it is recorded and reported, never ranked, so a cheaper result never displaces a stronger one.

The frontier. The *frontier* of B is the set of questions whose best result is not terminal — each carrying the route that produced its best result and that result’s progress evidence, which is exactly “the non-terminating results with the route to get there.” There is no second ledger: what the previous design kept as books, boards, and demand records is the frontier’s partial results, read as prose. *Expanding the frontier* means strictly improving some question’s best result — level, bound, or grade, never cost.

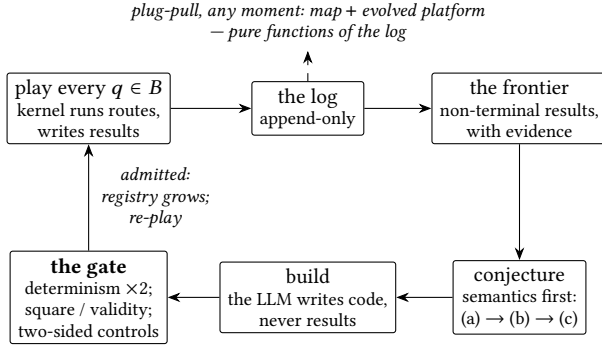


Figure 3. One loop iteration, no valve. The LLM conjectures and builds; only the kernel writes results, and only the gate writes the registry. The conjecture order (a)–(c) is fixed: semantics first, then syntax. Plug-pull is safe at any moment because both exit deliverables regenerate from the append-only log byte-identically, and best-per-question only ever improves.

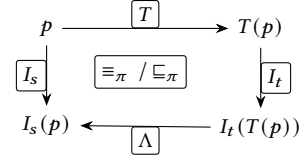
Monotonicity for free. The registry and the per-benchmark log are append-only, and best-per-question is a fold over the log. Exploration is therefore monotone *by construction*: appending records can only improve the best (best_mono), and a question once terminal stays terminal (terminal_ratchet) — both mechanized in the kernel’s own Lean development (Section 6). What the previous design carried as a facilitation theorem over a filtration model is now a property of a data structure. The exit deliverables inherit this: the frontier report is a pure function of the log and regenerates byte-identically, so pulling the plug mid-run loses nothing and forges nothing.

5 The loop: semantics first, then syntax

The LLM is presented a benchmark and runs autonomously until a human pulls the plug (Fig. 3): *play* every question along the feasible routes within budget — the kernel records the results; *read the frontier* — the non-terminal results with their evidence; *conjecture* what would move it; *build*; pass the build through the gate; re-play what the admission affects; repeat.

The conjecture order. Conjectures go *semantics first, then syntax* — this is the vision’s core discipline, and it is fixed, not a heuristic:

- (a) **new solving for existing languages** — a new solver pair, a portfolio or budget policy, a certificate printer, a discharge engine: new *decision procedures* first;
- (b) **new translation** — new pairs, new routes to existing solvers;
- (c) **new languages** — abstraction or specialization deltas, proposed only when a translation or solving move keeps winning ad hoc and reifying it would make



solves for	the discovered object	checked by
(a) the procedure	a solver pair	result validity
(b) T	a translation pair	the square, per run
(c) π	an abstraction	the lax square
(c) I_t	language + parent pair	the square
– I_s	reverse-engineering	external anchor

Figure 4. The conjecture order walks the square’s unknowns, semantics first. Solving for I_s — guessing the semantics of an existing real-world language — is the one inversion an internal check cannot adjudicate; it stays anchor-bound and outside the loop. A new language enters only as (c): stipulated semantics checked against a parent the platform already trusts, adding nothing to the trusted base.

the win reusable and cheap. New syntax is *earned* by demonstrated semantics, never invented ahead of it.

The order is the previous design’s closing insight made operational (Fig. 4): the commuting square $I_s(p) \equiv_{\pi} \Lambda(I_t(T(p)))$ relates four artifacts, and given any three, the equation constrains the fourth. Each conjecture class solves the square for a different unknown, and one gate adjudicates all of them, because *the gate checks the square, not the artifact*: it never asks which slot the untrusted producer filled.

One gate, no valve. Whatever is built — by the loop or by hand; manual registration is the same path, human-invoked, no special case — is admitted by one check: determinism is *measured* (every executable runs twice, outputs byte-compared), the square or result validity is checked on the entry’s corpus, and *two-sided controls* must hold — the intact artifact passes its vectors, and every supplied mutant fails, because a checker that cannot be made to fail is unfalsifiable. Admission evidence is stamped into the manifest by the checker, never self-reported. The previous design held registration behind a human valve; the kernel has none, and the reasoning is worth stating. What the valve protected — answer soundness — is protected by construction: existential certification bypasses route trust entirely, universal certification is graded, and the LLM has no code path to a result. What autonomy actually risks is registry clutter, which the result order neutralizes (junk never wins a best-per-question) and which pruning — a human act between runs — can clean; during a run the registry only grows.

Plug-pull. Stopping is safe at any moment: the driver checkpoints after every result and every admission, and the exit deliverables are pure functions of the log — (i) the *frontier summary*: best result, route, and cost per question, the

non-terminal results with their evidence, and the delta since iteration zero (expanded or not); (ii) if expanded, the *evolved* hurdy-gurdy: the kernel unchanged plus everything registered, with admission evidence. The next benchmark starts from it, which is how maps compound.

Bootstrap from empty. The kernel ships with zero languages and zero pairs, and everything above must work on that instance — the design’s simplicity test. Presented a benchmark, the LLM’s first act is writing the root language’s interpreter (the trusted base, graded *stipulated*; benchmark labels and supplied vectors are its only corroboration), then a first naive solver pair, then growth. Grades bottom out at *claimed* until the registry holds a second lineage — and the map says so, which is the honesty rule doing its work on day one.

6 What is proved, and what became of the theorems

The kernel’s own development. The kernel is the only hand-written code — five stdlib-only Python modules, under nine hundred lines — and its properties are mechanized in a dedicated, dependency-free Lean 4 development (kernel/mechanization/), separate from the previous calculus’s development, which remains the record of the old model:

- the result order on keys (*level, bound, grade*) is a strict order (Key.lt_irrefl, Key.lt_trans, Key.lt_asymm — axiom-free);
- best-per-question, a fold over the log, is monotone under append (best_mono): growing the log can only improve the best — the ratchet;
- once terminal, always terminal (terminal_ratchet): the frontier never re-opens a closed question;
- the grade ladder is strict and grade movement moves the key upward (ladder_strict, grade_moves_key — axiom-free).

The build prints each theorem’s trusted base; the ratchet lemmas rest on propext and Quot.sound only.

What is asked of generated code. Generated content is asked for proofs *when appropriate and feasible*: a proof obligation enters a manifest when the artifact is kernel-adjacent — a certificate schema must state (and, where feasible, prove) its fail-safe direction; a specialization’s embedding, its exactness; a new result payload, its ordering. Per-program translation correctness is deliberately *not* proved: that is what the square checks empirically, per run, which is the platform’s founding economy.

The six theorems, re-accounted. The previous design stated six facilitation theorems over a filtration model — in the prior version of this paper, preserved in the repository’s history beside the era record. The redesign keeps their intent

and relocates their burdens; the accounting is the cheapest way to see what the redesign gave up and what it kept:

was	became
F1 unfalsifiable map	by construction: only the kernel writes results; witnesses replay, universals grade, contradictions are recorded events
F2 monotone exploration	a data-structure property: append-only log + fold (best_mono, terminal_ratchet)
F3 complete local gradient	retired with its taxonomy: the typed diagnosis became the partial result’s evidence, read by an LLM, not a derivation engine
F4 relative completeness	retired with its assumptions (fair enumeration, gate liveness); what remains is measured, not proved
F5 saturation terminates	replaced by plug-pull: no fixpoint claim; exit deliverables are pure functions of the log at every moment
F6 domain-genericity	became bootstrap from empty: the kernel ships with zero languages, and the domain kit collapses to <i>write the root interpreter, pin the benchmark</i>

The two retirements are deliberate, and both have the same cause: F3 and F4 were theorems *about the demand machinery* — the five-condition filtration and the fair enumeration of its typed targets — and the machinery was removed because its consumer changed. What they bought (knowing what to build next, and that building it suffices) is now bought empirically, by the conjecturing LLM reading richer evidence than the taxonomy ever encoded — a trade of proved completeness for observed productivity that the status of Section 7 exists to keep honest.

7 The experiments this paper rests on

Two experiments substantiate this paper: the campaign that forced the redesign, and the redesigned platform’s first demonstration. Nothing else is claimed.

The campaign that forced the redesign. The previous design ran a pre-registered protocol to completion: seven loop iterations over the pinned 110-question hwmcc-sosylab-beem suite [19], every instance sha256-pinned, the ledger deposited in the repository and the full report regenerating from it byte-identically. The outcome, read from the ledger: 82 of 110 questions answered at least once and never receded; four proved at every depth; every verdict agreed with every external label; and, after the host’s engine enumeration closed, exactly *one* surviving demand entry — *build a better unbounded engine* — with 29 citing questions. The structural lesson mattered more than the numbers: every artifact the loop could admit was meaning-preserving by construction, so the loop was *conservative over truth* — seven iterations of a discovery loop produced instruments, never facts, and its one demand was exogenous.

That ceiling, together with the judgment that the demand machinery had grown past its justification, is what Section 1 designed away from.

The kernel, demonstrated by carry-over. The redesign’s first test is the previous platform itself: Era-3 code is not discarded but *mined* — wrapped in kernel manifests and re-certified through the gate it must now pass. The first slice is the BTOR2 interpreter (vectors from the existing test suite; a semantics-blind mutant as negative control) and the btormc solver pair (bounded model checker; witness carry-back; an always-*all* mutant; the canary discipline carried over: silent exhaustion is trusted only after the binary answers *sat* on a trivially reachable input). Both admitted: vectors pass, every mutant fails, determinism measured twice. The demonstration benchmark then plays three questions through the kernel, and its map is worth reading in full because every row exercises a different part of the design:

question	ask	best result	grade	status
counter-reach	$\exists, k=20$	witness, depth 5	replayed	terminal
frozen-unreach	$\forall, k=20$	all(20)	claimed	terminal
frozen-unreach	$\forall, \infty$	all(20)	claimed	frontier

The witness was solved at the target, carried back, and replayed through the source interpreter — terminal at the rung trust cannot raise or lower. The bounded universal is terminal but graded *claimed*, exactly as strong as its evidence: btormc prints no certificate, and the ladder says so rather than rounding up. And the unbounded ask sits honestly on the frontier at *all*(20), carrying its route and evidence — the campaign’s structural finding reproduced in miniature, and the first conjecture the order of Section 5 licenses is class (a): a second solver pair, an unbounded engine, whose disjoint lineage would also light *corroborated* for the bounded row. The remaining carry-over — thirteen languages, fifteen translation pairs, ten solver backends — is named work, deliberately of the shape the loop itself performs, and it is the redesign’s standing experiment: whether the conjecture order, fed by frontier evidence alone, reconstructs and then extends what the previous generation built by hand.

8 Related work

Decidability by reduction. Answering a question by carrying it to a decision procedure is a classical discipline, from quantifier elimination [23, 26] and interpretations that transfer decidability between theories [24, 27] to verification practice, where the move is institutional: bounded model checking reduces to SAT [4], theory combination composes procedures [18], intermediate verification languages pipeline programs to SMT [1, 10], and Ivy makes landing in a decidable fragment the design objective itself [20, 21]. Each strand fixes the reduction and asks what it decides; hurdy-gurdy makes the evolving, gated set of reductions the object of study, with budgeted cost inside the definition and the

boundary — the frontier of non-terminal results — as the deliverable.

Single-edge theories, abstraction, certificates. Certified compilation [12, 13] and translation validation [14, 17, 22] are statements about one translation; the companion calculus [11] develops the graph-level theory this paper consumes, and the kernel’s move is to run the graph as an explorer. Abstract interpretation [8] supplies the lax half of the directional square, and CEGAR [6] the shape of the refinement loop — with the structural difference that refinements are registered, reusable pairs and spurious counterexamples persist as frontier evidence. Proof-carrying code [16], certifying algorithms [15], verification witnesses [3], and checked proof formats [25, 28] supply the trust vocabulary; the grade ladder adds that the *absence* of a certificate is data — a claim graded *claimed* is a standing demand for the printer and discharge engine that would upgrade it.

Competitions; conjecture generation. HWMCC [19] and SV-COMP [2] rank solvers on fixed benchmarks; the loop inverts the roles — the benchmark is fixed and the *instrument* grows, the frontier exported as evidence-carrying open problems. Automated conjecture-making [7, 9] and theory exploration [5] filter candidates syntactically or statistically; here the LLM proposes only code and conjectures, and a deterministic kernel adjudicates every claim by running it — the producer stays a syntax engine, permanently, and that is the design rather than a concession.

9 Limitations and conclusion

Three walls stand at every registry size, and the design is drawn up to them, not past them. *Computability*: a route re-expresses a question, never changes its degree; unbounded universal claims rest on certificates, and without one they are bounded claims under declared bounds, labeled so. *Trust supply*: capability scales with generated pairs, but the trusted base is the root interpreters, graded stipulated until corroborated — and independent lineages, like external anchors before them, exist in finite supply, so the grade ladder records where trust saturates rather than pretending it does not. *Cost*: deciding is exponential in the worst case everywhere that matters; budgets are permanent provenance, capped is labeled capped, and a frontier that stops moving under a fixed registry is a finding — the measured practice-frontier of that registry — not a failure.

Within the walls, the claim is deliberately modest in kind: an architecture in which one edge kind carries both translation and solving, one gate admits everything, one currency — the result — holds both progress and failure, and one small proved kernel writes every entry, so that an autonomous, untrusted LLM can grow the instrument and play it without any answer resting on its word. The previous generation of the platform is the first content of the new one, re-certified

through the gate; the standing experiment is whether the conjecture order — procedures, then translations, then languages; semantics first, then syntax — fed by frontier evidence alone, reconstructs what was built by hand and then builds what was not. The loop turns until the player stops cranking; what remains is the map and the evolved instrument, and the next benchmark starts on the ground the last one won. The vision is the means. The map is the end.

Acknowledgments

This work was co-funded by the Czech Science Foundation under Grant No. 23-07580X and the European Union under the project Robotics and Advanced Industrial Production (reg. no. CZ.02.01.01/00/22_008/0004590).

References

- [1] Mike Barnett, Bor-Yuh Evan Chang, Robert DeLine, Bart Jacobs, and K. Rustan M. Leino. 2005. Boogie: A Modular Reusable Verifier for Object-Oriented Programs. In *Formal Methods for Components and Objects (FMCO) (LNCS, Vol. 4111)*. 364–387.
- [2] Dirk Beyer. 2021. Software Verification: 10th Comparative Evaluation (SV-COMP 2021). In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS, Vol. 12652)*. 401–422.
- [3] Dirk Beyer, Matthias Dangl, Daniel Dietsch, Matthias Heizmann, and Andreas Stahlbauer. 2015. Witness Validation and Stepwise Testification across Software Verifiers. In *ESEC/FSE*.
- [4] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. 1999. Symbolic Model Checking without BDDs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS, Vol. 1579)*. 193–207.
- [5] Koen Claessen, Nicholas Smallbone, and John Hughes. 2010. QuickSpec: Guessing Formal Specifications Using Testing. In *TAP*. 6–21.
- [6] Edmund M. Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. 2000. Counterexample-Guided Abstraction Refinement. In *CAV (LNCS, Vol. 1855)*. 154–169.
- [7] Simon Colton. 2002. *Automated Theory Formation in Pure Mathematics*. Springer.
- [8] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *POPL*. 238–252.
- [9] Siemion Fajtlowicz. 1988. On Conjectures of Graffiti. *Discrete Mathematics* 72, 1–3 (1988), 113–118.
- [10] Jean-Christophe Filliâtre and Andrei Paskevich. 2013. Why3 — Where Programs Meet Provers. In *ESOP*.
- [11] Christoph Kirsch. 2026. Untrusted Authors, Trusted Answers: A Calculus of Fidelity-Graded Translations. arXiv preprint arXiv:2607.14137, v2. Code, evidence, and the Lean mechanization: <https://github.com/cksystems/hurdy-gurdy> (v2 is repository tag arxiv.2).
- [12] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. 2014. CakeML: A Verified Implementation of ML. In *POPL*.
- [13] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* 52, 7 (2009), 107–115.
- [14] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: Bounded Translation Validation for LLVM. In *PLDI*.
- [15] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161.
- [16] George C. Necula. 1997. Proof-Carrying Code. In *POPL*.
- [17] George C. Necula. 2000. Translation Validation for an Optimizing Compiler. In *PLDI*.
- [18] Greg Nelson and Derek C. Oppen. 1979. Simplification by Cooperating Decision Procedures. *ACM Transactions on Programming Languages and Systems* 1, 2 (1979), 245–257.
- [19] Aina Niemetz, Mathias Preiner, Clifford Wolf, and Armin Biere. 2018. Btor2, BtorMC and Boolector 3.0. In *CAV*.
- [20] Oded Padon, Giuliano Losa, Mooly Sagiv, and Sharon Shoham. 2017. Paxos Made EPR: Decidable Reasoning about Distributed Protocols. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (2017), 108:1–108:31.
- [21] Oded Padon, Kenneth L. McMillan, Aurojit Panda, Mooly Sagiv, and Sharon Shoham. 2016. Ivy: Safety Verification by Interactive Generalization. In *PLDI*. 614–630.
- [22] Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. Translation Validation. In *TACAS*.
- [23] Mojżesz Presburger. 1929. Über die Vollständigkeit eines gewissen Systems der Arithmetik ganzer Zahlen, in welchem die Addition als einzige Operation hervortritt. In *Comptes Rendus du I Congrès de Mathématiciens des Pays Slaves*. 92–101.
- [24] Michael O. Rabin. 1969. Decidability of Second-Order Theories and Automata on Infinite Trees. *Trans. Amer. Math. Soc.* 141 (1969), 1–35.
- [25] Yong Kiam Tan, Marijn J. H. Heule, and Magnus O. Myreen. 2021. cake_lpr: Verified Propagation Redundancy Checking in CakeML. In *TACAS*.
- [26] Alfred Tarski. 1951. *A Decision Method for Elementary Algebra and Geometry* (2nd ed.). University of California Press.
- [27] Alfred Tarski, Andrzej Mostowski, and Raphael M. Robinson. 1953. *Undecidable Theories*. North-Holland.
- [28] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt Jr. 2014. DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs. In *SAT*.